

Änderungsbericht: Optimierung des HUD- und Kodex-Systems in ZEITRISS

Ausgangslage

Bei ersten Tests fiel auf, dass das Intro-HUD (z.B. „Willkommen, Agent. **Kodex-Sync aktiv ...**“) beim Start der Nullzeit-Mission

- **nicht als erste UI-Ebene angezeigt wird**, sondern der Kodex spricht zuerst.
- **als schwarzer Code-Block** statt als graues **Inline-HUD** mit Symbolen erscheint.

Dies widerspricht den Design-Richtlinien:

- In `systems/gameflow/cinematic-start.md` wird beschrieben, wie nach der Zeitreise die retina-Linse hochklappt und dem Agenten das HUD samt Koordinaten, Vitalwerten und temporaler Stabilität anzeigt; erst danach gibt der Kodex einen Tipp ¹.
- Das Dokument `characters/hud-system.md` betont, dass HUD-Overlays **als graue Inline-Backticks mit Icons** geliefert werden sollen und kurze, maximal sechswörtige Meldungen sind ².

Aktuell scheint die Reihenfolge im Code inkorrekt zu sein und die Formatierung wird als Codeblock ausgegeben. Ursache ist eine falsche Nutzung der Makros `kodex_hint_for_scene` bzw. `hud_tag` sowie die Reihenfolge in `runtime.js` oder der Makros für den Missionsstart. Die nachfolgenden Optimierungsschritte beheben diese Inkonsistenzen ohne ein Re-Design zu erfordern.

Übersicht der zu ändernden Module

Modul/Datei	Problem	Lösungsvorschlag
<code>systems/gameflow/cinematic-start.md</code>	Das Intro hat momentan die Form eines Kodex-Monologs. Der HUD-Toast „Willkommen, Agent. Kodex-Sync aktiv ...“ wird als Code-Block gerendert und nicht über den HUD-Overlay-Mechanismus ausgegeben.	Verwende innerhalb der Macro-Schicht das HUD-Makro statt Kodex-Makro: <code>hud_tag("Willkommen, Agent. Kodex-Sync aktiv ...", inline=true)</code> vor dem Kodex-Tipp. Entferne triple-backticks oder Code-Fence-Formatierungen, damit der Toast als graues Inline-HUD erscheint.

Modul/Datei	Problem	Lösungsvorschlag
Makro-Datei <code>internal/runtime/toolkit-runtime-makros.md</code> (bzw. <code>toolkit-runtime-makros.md</code>)	Der Makro <code>StartScene</code> ruft zwar <code>scene_overlay</code> , aber es gibt Stellen, an denen <code>kodex_hint_for_scene</code> separat aufgerufen wird (z.B. bei Cinematics). Dadurch kann der Kodex vor dem HUD sprechen.	Ergänze in den Makros <code>StartMission</code> / <code>StartScene</code> eine explizite HUD-Toast für die Nullzeit-Synchro (<code>hud_tag(hud_vocab.kodex_sync)</code>) bevor <code>kodex_hint_for_scene</code> ausgeführt wird. Alternativ richte einen neuen Makro <code>kodex_sync_intro()</code> ein, der zuerst <code>scene_overlay</code> und dann <code>kodex_hint_for_scene</code> aufruft.
<code>runtime.js</code>	Möglicherweise wird im Spielstart der Kodex-Toast mittels <code>hud_toast</code> ohne prioritäres Einblenden des Overlays ausgeführt. Der HUD-Toast erscheint daher erst als zweites Element und verliert die Formatierung.	Stelle sicher, dass beim Start von Missionen (<code>StartMission</code> , <code>PlayMission</code> oder im Cinematic-Start) die Funktion <code>scene_overlay</code> / <code>hud_tag</code> vor allen <code>hud_toast</code> - oder <code>kodex</code> -Ausgaben aufgerufen wird. Gib das <code>Willkommen, Agent</code> -Toast als HUD-Tag aus (mithilfe <code>hud_tag</code> und <code>hud_vocab.kodex_sync</code>) und definiere ein eindeutiges <i>Priority-Tag</i> (z.B. <code>ENTRY</code>), damit es nicht vom Budget-System unterdrückt wird.
<code>scripts/lint_hud_kodex.py</code>	Der Linter warnt vor veralteter Klammern-Syntax (<code>[HUD:...]</code>) und <code>\$</code> -Wrappern, jedoch führt er nicht dazu, dass Cinematic-Dateien mit Code-Fence-Blocks auffallen.	Ergänze im Linter einen Test, der Codeblock-Fences (<code>...</code>) innerhalb HUD-Texten detektiert und einen Fehler ausgibt. So wird verhindert, dass künftige Szenen versehentlich Code-Blöcke für HUD-Inhalte nutzen.

Detaillierte Schritte

1. HUD-Intro in `cinematic-start.md` korrigieren

- Entferne alle Formatierungen, die den Text als Codeblock ausgeben (z.B. triple-backticks).
- Nutze das Makro `hud_tag` im Fließtext, um das Intro-HUD auszugeben. Beispiel:

```
{% set intro_msg = "Willkommen, Agent. Kodex-Sync aktiv • Koordinaten:
{{ coords }} • Vitalwerte: {{ vitals }} • Stabilität: {{ stability }}" %}
{{ hud_tag(intro_msg) }}
{{ kodex_hint_for_scene("menü", "Verwende ‹Menü›-Befehl zum Öffnen des
taktischen Interfaces") }}
```

Der Agent sieht so zuerst das graue HUD (inklusive Icons aus `hud_tag`), danach den Kodex-Tipp.

2. Makro-Reihenfolge anpassen

- Öffne `internal/runtime/toolkit-runtime-makros.md` und suche nach Makros für den Missions-/Szenenstart (`StartMission`, `StartScene`, `StartCinematic` etc.).
- Füge in dem Makro, der das Cinematic startet, vor dem Kodex-Hint folgende Zeile ein:

```
{{ hud_tag(hud_vocab.kodex_sync) }}
```

Das `hud_vocab.kodex_sync` sollte in `hud_vocab` definiert sein (z.B. `kodex_sync: "Willkommen, Agent. Kodex-Sync aktiv"`). Wenn es dort fehlt, erweitere `hud_vocab` entsprechend.

- Stelle sicher, dass `scene_overlay` bzw. `StartScene` immer vor `kodex_hint_for_scene` aufgerufen wird. Wenn ein Cinematic eine eigene Reihenfolge hat, erstelle einen Wrapper-Makro `kodex_sync_intro()` wie folgt:

```
{% macro kodex_sync_intro() %}  
  {{ scene_overlay() }}  
  {{ hud_tag(hud_vocab.kodex_sync) }}  
  {{ kodex_hint_for_scene(current_scene.id) }}  
{% endmacro %}
```

und verwende diesen im Cinematic statt direkter Kodex-Ausgabe.

3. `runtime.js` auf Toast-Reihenfolge prüfen

- Durchsuche die Funktionen `StartMission`, `StartScene`, `PlayMission` und den Teil, der die Nullzeit-Szene startet. Achte darauf, dass **keine Kodex-Ausgabe** über `hud_toast` oder `emit_toast` vor dem HUD-Overlay erfolgt.
- Falls `emit_toast` mit Kodex-Text vor `scene_overlay` steht, verschiebe den `scene_overlay`-Aufruf nach oben.
- Du kannst auch `emit_toast` um ein optionales Argument `priority_tag` erweitern und dem Intro-Toast das Tag `ENTRY` geben, da dieses Tag laut `HUD_PRIORITY_TAGS` stets angezeigt wird (es wird vom Budget-System nicht unterdrückt). Ein Beispielaufruf:

```
hud_toast(state, "Willkommen, Agent. Kodex-Sync aktiv", { priority: true,  
tags: ["ENTRY"] });
```

So wird sichergestellt, dass der Einblenden-Toast trotz Budget-Limit angezeigt wird.

4. Linter anpassen

- In `scripts/lint_hud_kodex.py` eine neue Regex hinzufügen, die Code-Blocks innerhalb HUD-Texten erkennt. Beispiel:

```
code_block_pattern = re.compile(r"```.+?```", re.DOTALL)  
if code_block_pattern.search(line):
```

```
print(f"ERROR: Code-Block in HUD-Text entdeckt in {filename}:{ln}. Nutze  
hud_tag statt triple-Backticks.")
```

Dies stellt sicher, dass künftige Szenen keine Code-Fences verwenden.

5. Optional: Icons und graue Inline-Backticks prüfen

- Überprüfe, ob das Intro-HUD im Cinematic die nötigen Icons (z.B. Vitals, Stress, Paradoxon-Index) enthält. Wenn die Makros `hud_tag` und `scene_overlay` bereits diese Symbole generieren, ist keine weitere Anpassung nötig.
- Sollte das Cinematic eigene Symbole benötigen, erweitere `hud_tag` bzw. `hud_vocab` um passende Einträge.

Ergebnis und erwartetes Verhalten

Nach Umsetzung dieser Anpassungen erfolgt der Spielstart wie folgt:

1. Beim Start der Nullzeit-Mission ruft der Code das HUD-Overlay (`scene_overlay`) auf. Es erscheint in grauen Inline-Backticks mit den Standardfeldern `EP`, `MS`, `SC`, Modus, Mission, Gate, FS, Stress, Px-Bar, Level, usw.
2. Direkt danach blendet das HUD (nicht der Kodex) einen kurzen Toast „Willkommen, Agent. Kodex-Sync aktiv ...“ ein. Die Meldung erscheint inline, mit passenden Symbolen und in grauer Schrift, sodass sie als Teil des Interfaces wirkt.
3. Erst im Anschluss spricht der Kodex über das Comlink und liefert einen Tipp (z.B. wie man das Menü öffnet). Der Kodex-Output ist bewusst länger und befindet sich unter dem HUD-Toast, sodass die diegetische Immersion bestehen bleibt.
4. Das Budget-System im `runtime.js` unterdrückt dadurch nicht den wichtigen Intro-Toast, da er als `ENTRY`-Tag markiert ist. Weitere HUD-Meldungen in der Szene folgen den definierten Limits.
5. Dank des Linter-Updates werden versehentliche Code-Blocks in HUD-Texten frühzeitig erkannt.

Mit diesen Änderungen harmonisieren HUD und Kodex erneut und erfüllen die Design-Vorgaben aus `characters/hud-system.md` ² und den Cinematic-Anweisungen ¹. Ein chirurgischer Eingriff in wenigen Modulen reicht aus, um das Benutzererlebnis zu verbessern.

¹ cinematic-start.md

<https://github.com/pchospital-lab/ZEITRISS-md/blob/7fe82225ff6b0984f1638d09d3e2e85a0795b23d/systems/gameflow/cinematic-start.md>

² hud-system.md

<https://github.com/pchospital-lab/ZEITRISS-md/blob/7fe82225ff6b0984f1638d09d3e2e85a0795b23d/characters/hud-system.md>